

Classical Machine Learning

Week 0

Plan

- Setting up your learning and programming environment

Getting started

- [Setting up your ML environment \(Setup_NYU.ipynb\)](#)
 - [Choosing an ML environment](#)
[\(Choosing an ML Environment NYU.ipynb\)](#)
- [Quick intro to the tools \(Getting_Started.ipynb\)](#)

Week 1

Plan

We give a brief introduction to the course.

We then present the key concepts that form the basis for this course

- For some: this will be review

Intro to Advanced Course

- [Introduction to Advanced Course \(Intro_Advanced.ipynb\)](#)

Using an AI Assistant as a Personal Tutor

Knowledge of Deep Learning is a prerequisite for this course.

For those of you who need a review, we will do so at a **very rapid pace and abbreviated manner**.

But using AI Assistants (e.g., ChatGPT) can be a great resource for getting you up to speed.

The key is to using them as a personal tutor. Some advice

- describe the role you want them to play (e.g., Professor, practitioner)
 - this sets the level for depth of knowledge it will convey
- describe your level of knowledge (Graduate student, undergraduate, hacker)
 - this sets the level of complexity for the responses it provides
- Treat it as a tutor
 - ask for a concept to be explained, at varying levels of depth
 - ask follow-up questions until you get what you need

[Here \(https://www.perplexity.ai/search/you-are-an-expert-in-deep-learningPHF81eAScGyAJogE5idCw?0=\)](https://www.perplexity.ai/search/you-are-an-expert-in-deep-learningPHF81eAScGyAJogE5idCw?0=) is an example of such a conversation using Perplexity as the AI Assistant

- you may use any Assistant that you prefer: they are very similar in capabilities

Review/Preview of concepts from Intro Course (very abbreviated)

Here is a *quick reference* of key concepts/notations from the Intro course

- For some: it will be a review, for others: it will be a preview.
- We will devote a sub-module of this lecture to elaborate on each topic in slightly more depth.
 - For a more detailed explanation: please refer to the material from the Intro course ([repo \(https://github.com/kenperry-public/ML_Spring_2023\)](https://github.com/kenperry-public/ML_Spring_2023))
- [Review and Preview \(Review_Advanced.ipynb\)](#).

You may want to run your code on Google Colab in order to take advantage of powerful GPU's.

Here are some useful tips:

[Google Colab tricks \(Colab_practical.ipynb\)](#).

Transformers: Review

Preview

There is lots of interest in Large Language Models (e.g., ChatGPT). These are based on an architecture called the Transformer. We will introduce the Transformer and demonstrate some amazing results achieved by using Transformers to create Large Language Models.

Attention is a mechanism that is a core part of the Transformer. We will begin by first introducing Attention.

We will then take a detour and study the Functional model architecture of Keras. Unlike the Sequential model, which is an ordered sequence of Layers, the organization of blocks in a Functional model is more general. The Advanced architectures (e.g., the Transformer) are built using the Functional model.

Once we understand the technical prerequisites, we will examine the code for the Transformer.

[Transformers: Review \(Review Transformer.ipynb\)](#)

Suggested reading

- Attention
 - [Attention is all you need \(https://arxiv.org/pdf/1706.03762.pdf\)](https://arxiv.org/pdf/1706.03762.pdf)
- Transfer Learning
 - [Sebastian Ruder: Transfer Learning \(https://ruder.io/transfer-learning/\)](https://ruder.io/transfer-learning/)
- HuggingFace course
 - [Transformers: concepts \(https://huggingface.co/learn/nlp-course/chapter1/4?fw=pt\)](https://huggingface.co/learn/nlp-course/chapter1/4?fw=pt)

Further reading

- Attention
 - [Neural Machine Translation by Jointly Learning To Align and Translate](https://arxiv.org/pdf/1706.03762.pdf)

The remaining "reviews" will be omitted in class

- we will reference them in passing when dealing with the related topic in future lectures

Transformer: flavors (Transformer.ipynb#Transformer-variants)

Suggested reading

- HuggingFace course
 - Transformer styles (<https://huggingface.co/learn/nlp-course/chapter1/5?fw=pt>).

Attention: in depth

- Implementing Attention ([Attention Lookup.ipynb](#)).

Week 2: Technical tools/background

Functional Models

Plan

Enough theory (for the moment) !

The Transformer (whose theory we have presented) is built from plain Keras.

Our goal is to dig into the **code** for the Transformer so that you too will learn how to build advanced models.

Before we can do this, we must

- go beyond the Sequential model of Keras: introduction to the Functional model
- understand more "advanced" features of Keras: customomizing layers, training loops, loss functions
- The Datasets API

Basics

We start with the basics of Functional models, and will give a coding example of such a model in Finance.

- [Functional API \(Functional Models.ipynb\)](#)

Suggested reading

- Keras docs
 - [Functional API \(https://keras.io/guides/functional_api/\)](https://keras.io/guides/functional_api/)
 - [Making new layers and models via sub-classing \(https://keras.io/guides/making_new_layers_and_models_via_subclassing/\)](https://keras.io/guides/making_new_layers_and_models_via_subclassing/)

Functional Model Code: A Functional model in Finance: "Factor model"

We illustrate the basic features of Functional models with an example

- does not use the additional techniques of the next section (Advanced Keras)

[Autoencoders for Conditional Risk Factors](#)

[\(Autoencoder for conditional risk factors.ipynb\)](#)

- [code \(https://github.com/stefan-jansen/machine-learning-for-trading/blob/main/20 autoencoders for conditional risk factors/06 conditional a](https://github.com/stefan-jansen/machine-learning-for-trading/blob/main/20%20autoencoders%20for%20conditional%20risk%20factors/06%20conditional%20autoencoders%20for%20conditional%20risk%20factors.ipynb)

Suggested reading

- [Autoencoder asset pricing models \(https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3335536\)](https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3335536)

Threading

In our [Autoencoders for Conditional Risk Factors](#)

[\(Autoencoder for conditional risk factors.ipynb\)](#), we skipped over an important detail

Putting it all together: Code: the Transformer (continued)

Plan

With the base of advanced Keras under our belts, it's time to understand the Transformer, in code

We will examine the code in the excellent [TensorFlow tutorial on the Transformer](https://www.tensorflow.org/text/tutorials/transformer) (<https://www.tensorflow.org/text/tutorials/transformer>)

- more in-depth than our presentation
- more background

The tutorial is especially recommended for those without the basics of the Transformer from my Intro course

- [The Transformer: Understanding the Pieces](#) ([Transformer Understanding the Pieces.ipynb](#))
- [The Transformer: Code](#) ([Transformer code.ipynb](#))
- [Implementing Attention: detail](#) ([Implementing Attention.ipynb](#))
- [Choosing a Transformer architecture](#) ([Transformer Choosing a PreTrained Model.ipynb](#))

Suggested reading

There is an excellent tutorial on Attention and the Transformer which I recommend:

- [Tensorflow tutorial: Neural machine translation with a Transformer and Keras](https://www.tensorflow.org/text/tutorials/transformer) (<https://www.tensorflow.org/text/tutorials/transformer>).

Deeper dive

- [Implementing Attention](#) ([Attention Lookup.ipynb](#)).
- [Residual connections](#) ([RNN Residual Networks.ipynb](#)).

Week 3: Technical tools (continued)/Transfer Learning/HuggingFace

Plan

We continue with our introduction to the technical tools.

- [The Transformer: Code \(continued\) \(Transformer_code.ipynb#Training\)](#).
- [Choosing a Transformer architecture \(Transformer.ipynb#Use-cases-for-each-variant-of-Transformer\)](#).
- [Implementing Attention: detail \(Implementing_Attention.ipynb\)](#).

Advanced Keras (Deeper dive)

We will not cover this [notebook \(Keras_Advanced.ipynb\)](#) in class

- most of the material will be introduced as part of our study of different interesting models
- but this notebook is one convenient place to see them all
- consider it as a **reference** that collects multiple techniques in one place

Deeper dives

If you *really* want to dig into the micro-details of TensorFlow, here are some important subtleties

- [Computation Graphs \(Computation_Graphs.ipynb\)](#)
- [Eager vs Graph Execution \(TF_Graph.ipynb\)](#)

Inspecting a HuggingFace model from within Jupyter

Plan

We will be using models (including Transformers) hosted on the HuggingFace model hub.

Let's examine one in detail.

You don't need access to the source code ! You can inspect a model solely within Jupyter.

Our immediate focus is on the Transformer code; we will re-visit this notebook in more detail later to explain the HuggingFace "eco-system".

- [linked notebook: Using a pretrained Sequence Classifier \(https://colab.research.google.com/github/kenperry-public/ML_Advanced_Spring_2025/blob/master/HF_quick_intro_to_models.ipynb\)](https://colab.research.google.com/github/kenperry-public/ML_Advanced_Spring_2025/blob/master/HF_quick_intro_to_models.ipynb)
(Google Colab)

Transfer Learning

- [Transfer Learning: Review \(Review TransferLearning.ipynb\)](#)

Modern Transfer Learning: Model Hubs/HuggingFace

Plan

Modern Transfer Learning is often about

- Fine-tuning a pre-trained model

Pre-trained models are often collected in Model Hubs.

The hub we will use for the final project: HuggingFace

- illustrate how to fine-tune a pre-trained model
- quick Intro to HF
 - best way to learn: through the course !
 - uses Datasets
 - will introduce later
 - PyTorch version (uses Trainer); we will focus on Tensorflow/Keras version

HuggingFace Transformers course

The best way to understand and use modern Transfer Learning is via the [HuggingFace course \(https://huggingface.co/course\)](https://huggingface.co/course).

You will learn

- about the Transformer
- how to use HuggingFace's tools for NLP (e.g., Tokenizers)
- how to perform common NLP tasks
 - especially with Transformers
- how to fine-tune a pre-trained model
- how to use the HuggingFace dataset API

All of this will be invaluable for the Course Project.

- does not have to be done using HuggingFace
- but using at least parts of it will make your task easier
- [HuggingFace intro \(Transfer Learning_HF.ipynb\)](#)
 - [linked notebook: Using a pretrained Sequence Classifier \(HF quick intro to models.ipynb\)](#) (**local machine**)
 - [linked notebook: Using a pretrained Sequence Classifier](#)

Datasets: Big data in small memory

Plan

The Dataset abstraction provides a way to consume large datasets using a limited amount of memory.

We do a quick intro to HuggingFace datasets and how to use them with TensorFlow.

We subsequently introduce the TensorFlow Dataset (TFDS) API.

This is the last piece of technical info to enable the Final Project.

Background

- [Python generators \(Generators.ipynb\)](#)

HuggingFace Datasets

- how to use a Dataset
- using a HuggingFace dataset with Tensorflow

[HuggingFace datasets in action \(https://huggingface.co/learn/nlp-course/chapter3/2?fw=tf\)](https://huggingface.co/learn/nlp-course/chapter3/2?fw=tf)

TensorFlow Dataset (TFDS)

- [TensorFlow Dataset \(TF Data API.ipynb\)](#)

Notebooks

The following notebook is an exploration of the subtleties of TFDS (and TensorFlow)

It is particularly relevant to **user-defined functions**.

Most important conclusion

- the operations in a user-defined function should be *TensorFlow operations*
 - **not** Python operations
 - e.g., Image manipulation
 - use `tf.io` rather than `Python io.BytesIO`
 - use `tf.image` rather than `Python PIL.image`
- TensorFlow needs to convert your function to a Computation Graph
 - many Python operations cannot be converted (or at least not what you expect)
 - differentiable !
- [Dataset API: play around \(TFDatasets_play_v1.ipynb\)](#)

Suggested reading

- HuggingFace course

Week 4: Topics in Fine Tuning (Transfer Learning)/Synthetic Data

Fine Tuning at low cost

Parameter Efficient Transfer Learning

Transfer learning may be the "future of Deep Learning" in that we can adapt models (that are too big for us to train on our own) to our own tasks.

But Fine-Tuning a Pre-Trained model involves training a lot of parameters when the base model is large.

This may be difficult for a variety of reasons.

Can we adapt a Pre-Trained base model to a new task *without* training a large number of parameters ?

- [Parameter Efficient Transfer Learning \(ParameterEfficient_TransferLearning.ipynb\)](#)

Suggested reading

- Parameter Efficient Transfer Learning - article
(<https://lightning.ai/pages/community/article/understanding-llama-adapters/>)
- LoRA:Low Rank Adaptation of Large Language Models
(<https://arxiv.org/pdf/2106.09685.pdf>).
- Adapters
 - Parameter Efficient Transfer Learning for NLP
(<https://arxiv.org/pdf/1902.00751.pdf>)
 - LLM Adapters (<https://arxiv.org/pdf/2304.01933.pdf>)

Additional reading

- Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning (<https://arxiv.org/abs/2012.13255>).
- LoRA Learns Less and Forgets Less (<https://arxiv.org/pdf/2405.09673>).

Fine Tuning by Proxy

Can we fine-tune a large model

- **without** adapting its weights
- by fine-tuning a **small** model
 - much lower cost than fine-tuning a large model

Fine Tuning at low cost (continued)

Rather than having one model for every task:

- Is it possible to create a *single model* to solve every task ?

Text to text is a "Universal API"

- [Universal API \(LLM Universal API.ipynb\)](#)

Fine Tuning by Prompt Engineering

Given that an LLM can implement a Universal API

- How do we create the *best* prompt to cause an LLM to solve a new "target" task ?

We show two approaches

- one as an optimization problem

Fine-Tuning "without fine-tuning" learning a new task from exemplars

Building on the success of the Universal API

- Using a clever prompt design to solve any task
- Can we adapt a base LLM to solve a new Target task just by changing the prompt ?
- *without* further training (Fine-Tuning)

The answer is: yes !

All we need to do is

- show instances of examples for the new task *at inference time*
- as part of the prompt
- and the base model will "learn" to solve a new "target" task

This is called *In-Context Learning*.

[In-Context Learning\(In Context Learning.ipynb\)](#)

Topics in Synthetic Data

New major topic: Synthetic data.

After last week's "code-heavy" modules, we are back to "theory" !

We will address several ways to create new examples, starting with the simplest model and moving on to models that are more complex.

We wrap up by demonstrating a new trend

- using Large Language Models
- to create training data
- to improve Large Language Models !

Synthetic Data: Autoencoders

Generating synthetic data using Autoencoders and its variants.

"Vanilla" Autoencoder

[Autoencoder \(Autoencoders Generative.ipynb\)](#)

Suggested Reading

[TensorFlow Tutorial on Autoencoders](https://www.tensorflow.org/tutorials/generative/autoencoder)
(<https://www.tensorflow.org/tutorials/generative/autoencoder>)

Variational Autoencoder (VAE)

We now study a different type of Autoencoder

- that learns a *distribution* over the training examples
- by sampling from this distribution: we can create synthetic examples

[Variational Autoencoder \(VAE\) \(VAE_Generative.ipynb\)](#)

Suggested Reading

[TensorFlow tutorial on VAE](https://www.tensorflow.org/tutorials/generative/cvae) (<https://www.tensorflow.org/tutorials/generative/cvae>)

Further reading

[Tutorial on

Week 5: Transformers advanced

Multi-modal LLM's

Large Language Models were originally conceived to process Language (sequences of tokens of Natural Language).

But there is also interest in AI Assistants being able to handle *other types* of data

- image
- video -audio

Do we need new model types for these non-language data ?

We show how to adapt a Large Language Model (LLM) to understand (and generate) images.

Vector Quantized Autoencoders

While we are on the topic of Autoencoders, we present the Vector Quantized Autoencoder.

It is not so much a tool for creating Synthetic Data as a natural continuation of our Autoencoder exploration.

[Vector Quantized Autoencoder \(VQ_VAE_Generative.ipynb\)](#)

Suggested Reading

[vanilla VQ-VAE \(https://arxiv.org/pdf/1711.00937.pdf\)](#)

[VQ-VAE-2 paper \(https://arxiv.org/pdf/1906.00446.pdf\)](#)

DALL-E: Mixing Text and Image

We make use of the Quantized VAE technique we learned in the module on Autoencoders to enable us to mix text and image.

We discuss how a Text to Image model (convert the textual description of an image to an actual image) works.

- [CLIP \(CLIP.ipynb\)](#)
 - [Zero shot learning, prompt engineering Colab notebook: PyTorch \(https://github.com/openai/CLIP/blob/main/notebooks/Prompt_Engineering.ipynb\)](#)
- [DALL-E \(DALL-E.ipynb\)](#)
- [Vision Transformer \(Vision_Transformer.ipynb\)](#)

Suggested Reading

- [CLIP paper](https://openai.com/research/clip)
([https://cdn.openai.com/papers/Learning Transferable Visual Models From Natural Language Supervision](https://cdn.openai.com/papers/Learning_Transferable_Visual_Models_From_Natural_Language_Supervision.pdf))
- [DALL-E paper](https://arxiv.org/pdf/2102.12092.pdf) (<https://arxiv.org/pdf/2102.12092.pdf>)
- [OpenAI DALL-E 2 announcement](https://openai.com/dall-e-2/) (<https://openai.com/dall-e-2/>)
- [Vision Transformer](https://arxiv.org/pdf/2010.11929.pdf) (<https://arxiv.org/pdf/2010.11929.pdf>)
- [LiT paper](https://arxiv.org/pdf/2111.07991.pdf) (<https://arxiv.org/pdf/2111.07991.pdf>)

Contrastive Learning Objective

CLiP used a Contrastive Learning Objective

- find embeddings to
 - maximize similarity between related examples (e.g., image and the text that describes it)
 - minimize similarity between unrelated examples (e.g., image and text that *does not* describe it).

CLiP used Binary Cross Entropy to meet the objective; we study a different approach.

[Creating Embeddings for Similarity](#) ([Embeddings_similarity.ipynb](#))

Suggested Reading

Scaling (training-time, inference-time)

We now have the capabilities to build models with extremely large number of weights. Is it possible to have too many weights ?

Yes: weights, number of training examples and compute capacity combine to determine the performance of a model.

There is an empirical result that suggests that in order to take advantage of GPT-3's use of 175 billion weights

- 1000 times more compute is required than what was used
- 10 times more training examples is required compared to what was used

Here is a view of the historical evolution of model sizes, and what the future might hold

- [Large Language Model sizes: GPT history \(NLP Large Language Models.ipynb\)](#).
- [How large should my Transformer be ? \(Transformers Scaling.ipynb\)](#).

Suggested reading

- [Scaling laws \(https://arxiv.org/pdf/2001.08361.pdf\)](https://arxiv.org/pdf/2001.08361.pdf).

Further reading

- Inference budget
 - Transformer Inference Arithmetic (<https://kipp.ly/transformer-inference-arithmetic/>).
 - LLaMA: Open and Efficient Foundation Language Models (<https://arxiv.org/pdf/2302.13971.pdf>).
 - Large language models aren't trained enough (<https://finbarr.ca/lms-not-trained-enough/>).

Test-time compute

Plan

Scaling by using more inference-time compute

- [Scaling via Test-time compute \(Test_time_compute.ipynb\)](#)

Suggested reading

- [Large Language Monkeys: Scaling Inference Compute with Repeated Sampling \(https://arxiv.org/pdf/2407.21787\)](#)
- [Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters \(https://arxiv.org/pdf/2408.03314v1\)](#)

Reasoners

- [Reasoning \(LLM Thinking.ipynb\)](#)
- [Reasoning in Latent Space \(LLM Continuous Reasoning.ipynb\)](#)

Suggested reading

Advanced Topics

Non-differentiable operations

In order for Gradient Descent to find optimal parameter weights, all operations in the NN must be differentiable.

But there is a trick to allow non-differentiable operations to appear in an NN: Straight Through Estimation

[Straight Through Estimation \(VQ_VAE_Generative.ipynb#Quantization-is-not-differentiable\)](#).

In-Context Learning: Theory

Why does In_Context Learning work ? Some theories.

[In Context Learning: Theory \(In Context Learning Theory.ipynb\)](#)

Social Concerns

Alignment

Language models show great capabilities but also the potential for harm: biased and offensive generated text, for example. Can we "align" a model's output with human values

Additional Deep Learning resources

Here are some resources that I have found very useful.

Some of them are very nitty-gritty, deep-in-the-weeds (even the "introductory" courses)

- For example: let's make believe PyTorch (or Keras/TensorFlow) didn't exist; let's invent Deep Learning without it !
 - You will gain a deeper appreciation and understanding by re-inventing that which you take for granted

[Andrej Karpathy course: Neural Networks, Zero to Hero \(https://karpathy.ai/zero-to-hero.html\)](https://karpathy.ai/zero-to-hero.html)

- PyTorch
- Introductory, but at a very deep level of understanding
 - you will get very deep into the weeds (hand-coding gradients !) but develop a deeper appreciation

fast.ai

`fast.ai` is a web-site with free courses from Jeremy Howard.

- PyTorch
- Introductory and courses "for coders"
- Same courses offered every few years, but sufficiently different so as to make it worthwhile to repeat the course !
 - [Practical Deep Learning](https://course.fast.ai/) (<https://course.fast.ai/>)
 - [Stable diffusion](https://course.fast.ai/Lessons/part2.html) (<https://course.fast.ai/Lessons/part2.html>)
 - Very detailed, nitty-gritty details (like Karpathy) that will give you a deeper appreciation

Stefan Jansen: Machine Learning for Trading (<https://github.com/stefan-jansen/machine-learning-for-trading>)

An excellent github repo with notebooks

- using Deep Learning for trading
- Keras
- many notebooks are cleaner implementations of published models

Transformers advanced: Scaling

We now have the capabilities to build models with extremely large number of weights. Is it possible to have too many weights ?

Yes: weights, number of training examples and compute capacity combine to determine the performance of a model.

There is an empirical result that suggests that in order to take advantage of GPT-3's use of 175 billion weights

- 1000 times more compute is required than what was used
- 10 times more training examples is required compared to what was used

Here is a view of the historical evolution of model sizes, and what the future might hold

- [Large Language Model sizes: GPT history \(NLP Large Language Models.ipynb\)](#)
- [How large should my Transformer be ? \(Transformers Scaling.ipynb\)](#)

Suggested reading

- [Scaling laws \(https://arxiv.org/pdf/2001.08361.pdf\)](https://arxiv.org/pdf/2001.08361.pdf)

Further reading

- Inference budget
 - Transformer Inference Arithmetic (<https://kipp.ly/transformer-inference-arithmetic/>).
 - LLaMA: Open and Efficient Foundation Language Models (<https://arxiv.org/pdf/2302.13971.pdf>).
 - Large language models aren't trained enough (<https://finbarr.ca/lms-not-trained-enough/>).

Assignments

Your assignments should follow the [Assignment Guidelines](#)
([assignments/Assignment_Guidelines.ipynb](#)).


```
In [1]: print("Done")
```

Done

